

Grad-CAM for "Cat"

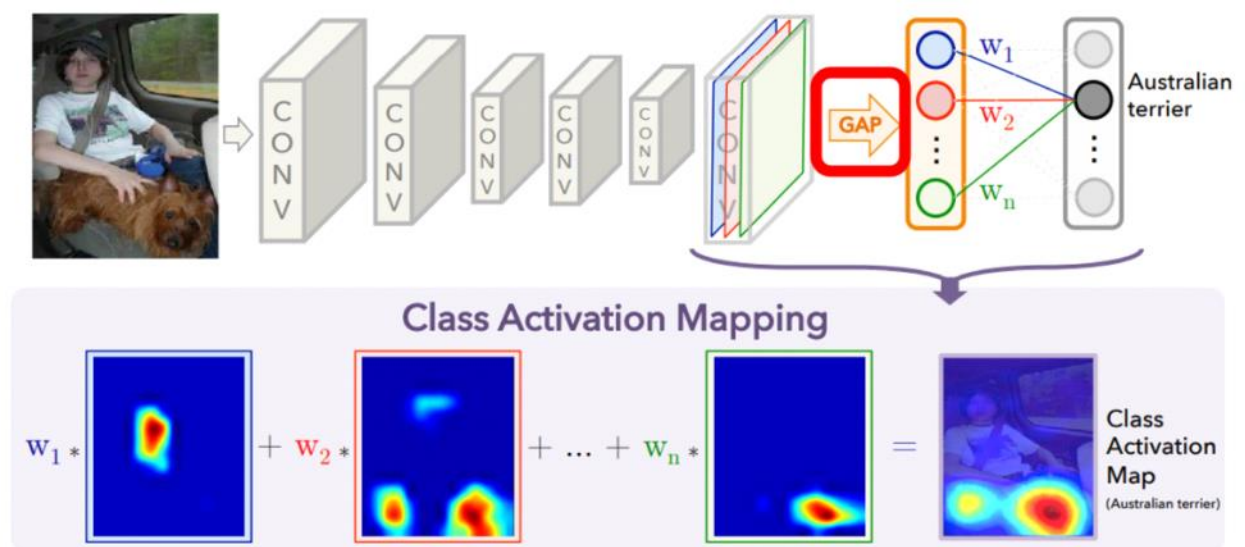


Grad-CAM for "Dog"

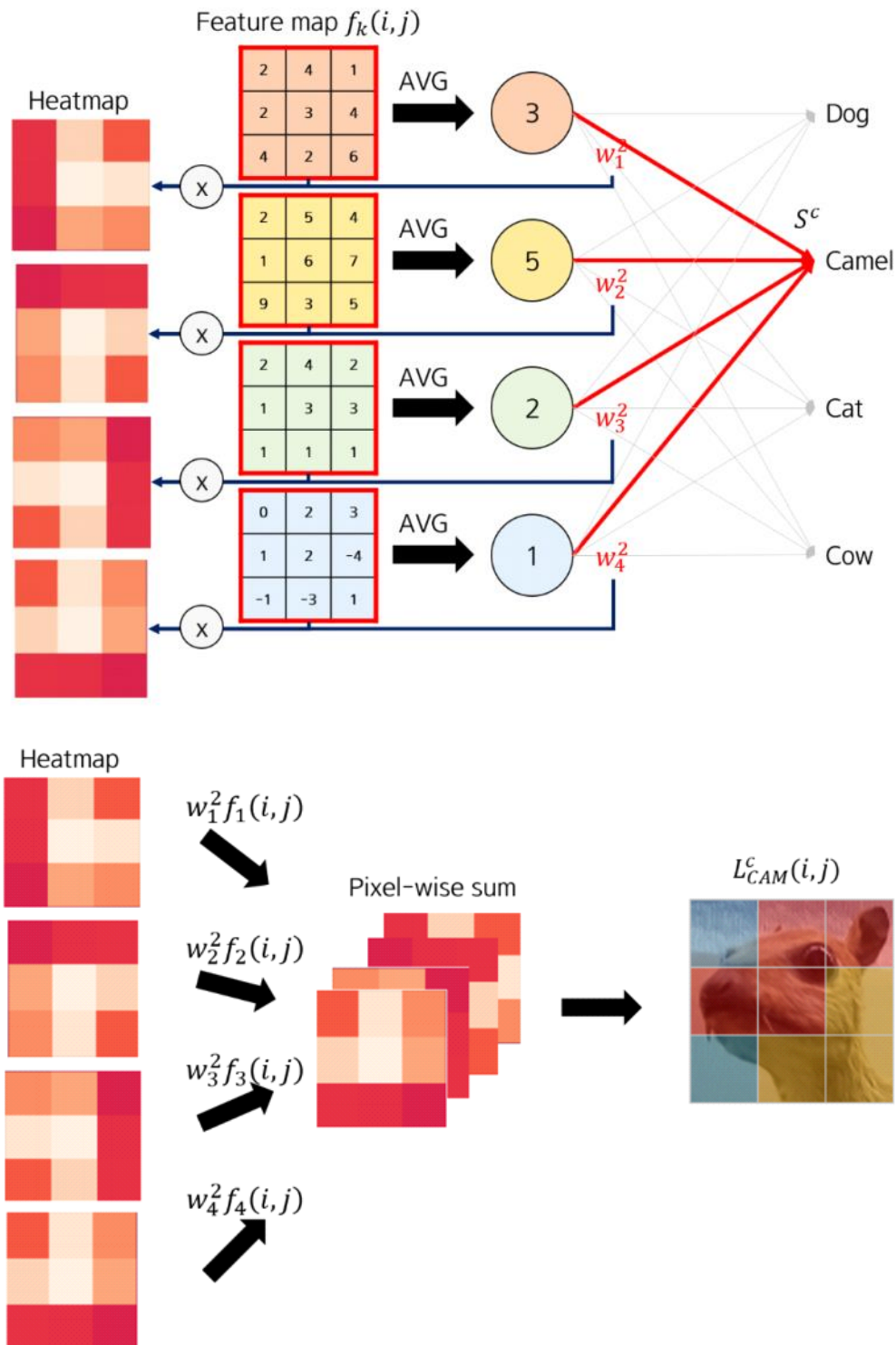


Class Activation Mapping(CAM)

: 컨볼루션 신경망(CNN)에서 특정 클래스의 예측에 기여하는 입력 이미지의 영역을 시각화하는 기법



$$L_{CAM}^c(i, j) = \sum_k w_k^c f_k(i, j)$$



```

features_blobs = []

def hook_feature(module, input, output):
    features_blobs.append(output.detach().cpu())

def generate_cam(feature_conv, weight_fc, class_idx):
    nc, h, w = feature_conv.shape
    cam = weight_fc[class_idx].dot(feature_conv.reshape((nc, h * w)))
    cam = cam.reshape(h, w)
    cam = cam - np.min(cam)
    cam = cam / np.max(cam)
    cam = np.uint8(255 * cam)
    return cv2.resize(cam, (224, 224))

model = models.resnet18(pretrained=True)
model.eval()
finalconv_name = 'layer4'
model._modules.get(finalconv_name).register_forward_hook(hook_feature)
params = list(model.parameters())
weight_softmax = params[-2].detach().cpu().numpy()

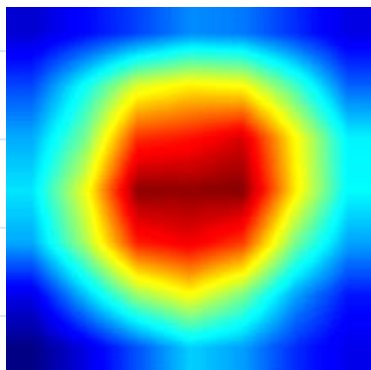
img_path = 'VAN_CAT.png'
input_tensor, raw_image = preprocess_image(img_path)

output = model(input_tensor)
probs = F.softmax(output, dim=1)
class_idx = torch.argmax(probs).item()

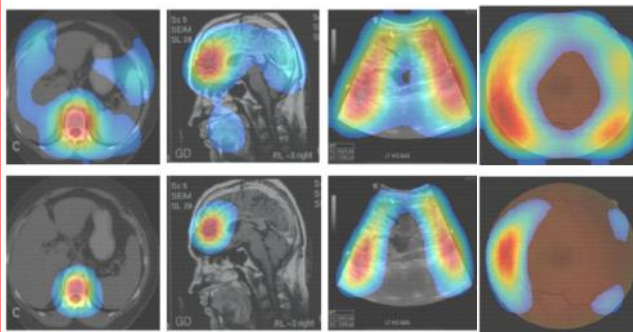
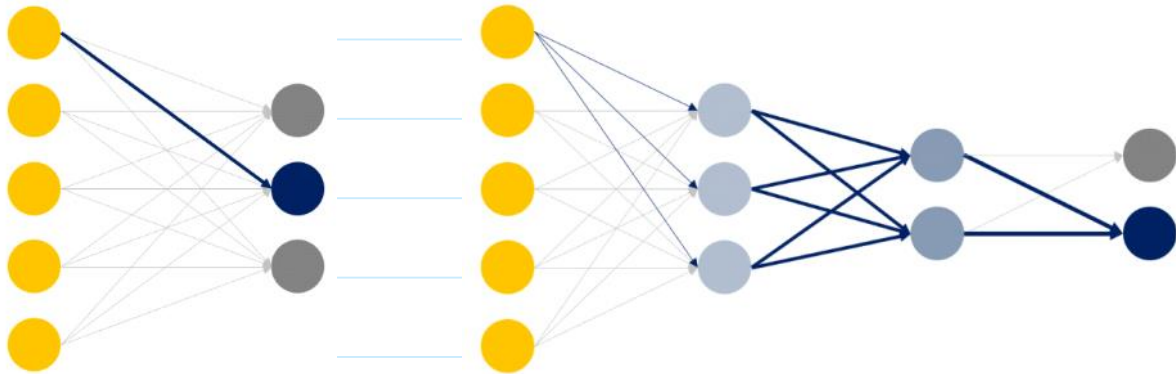
CAM = generate_cam(features_blobs[0].squeeze(0).numpy(), weight_softmax, class_idx)

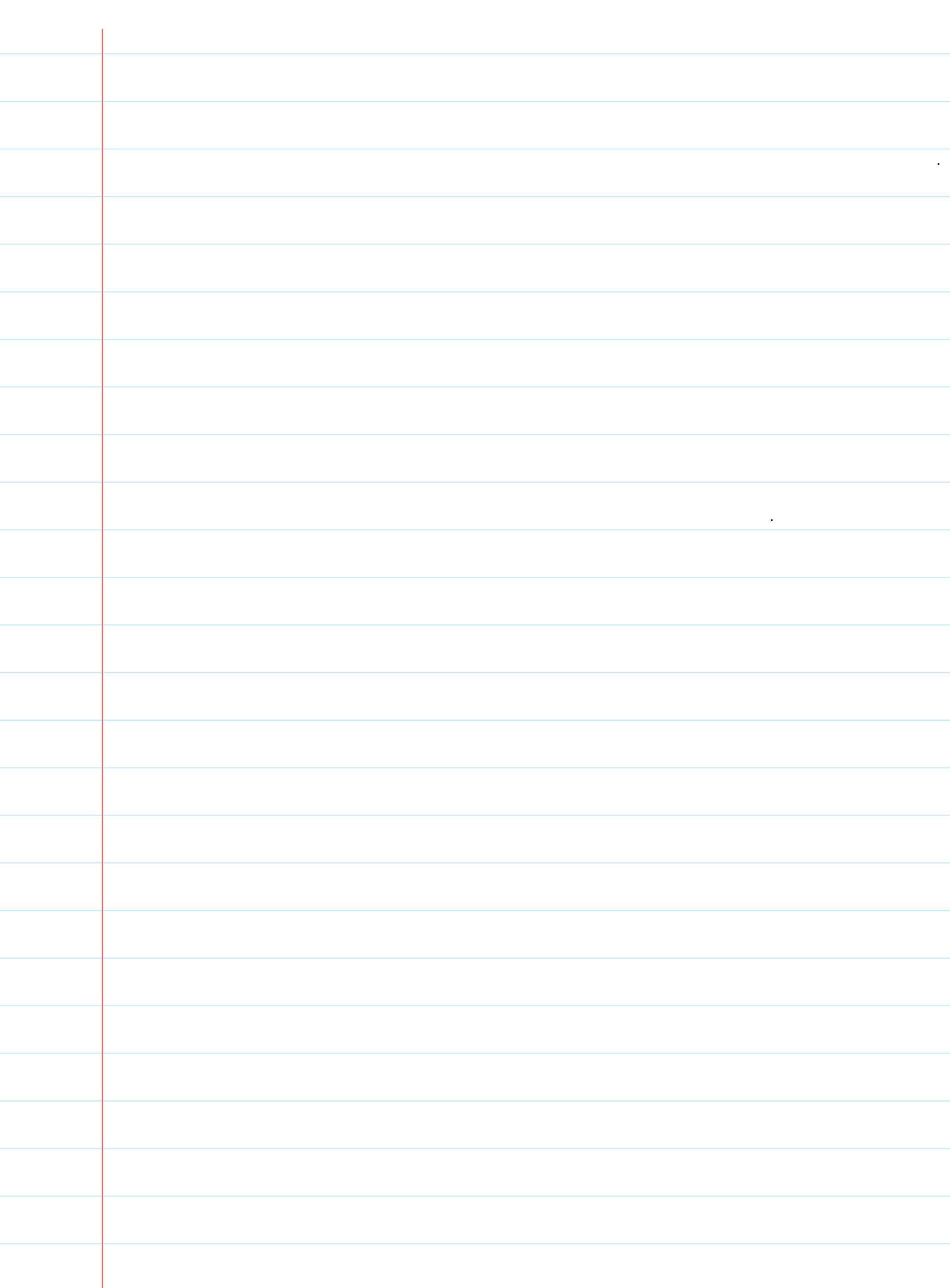
cam_heatmap = cv2.applyColorMap(CAM, cv2.COLORMAP_JET)
original = np.array(raw_image.resize((224, 224)))
overlay = cv2.addWeighted(original, 0.5, cam_heatmap, 0.5, 0)

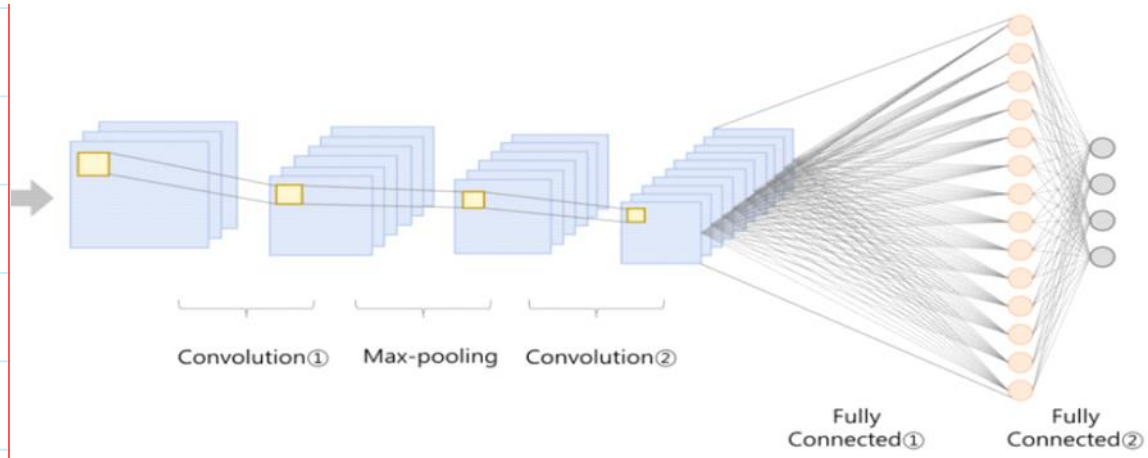
```



Gradient-weighted Class Activation Mapping(Grad-CAM)

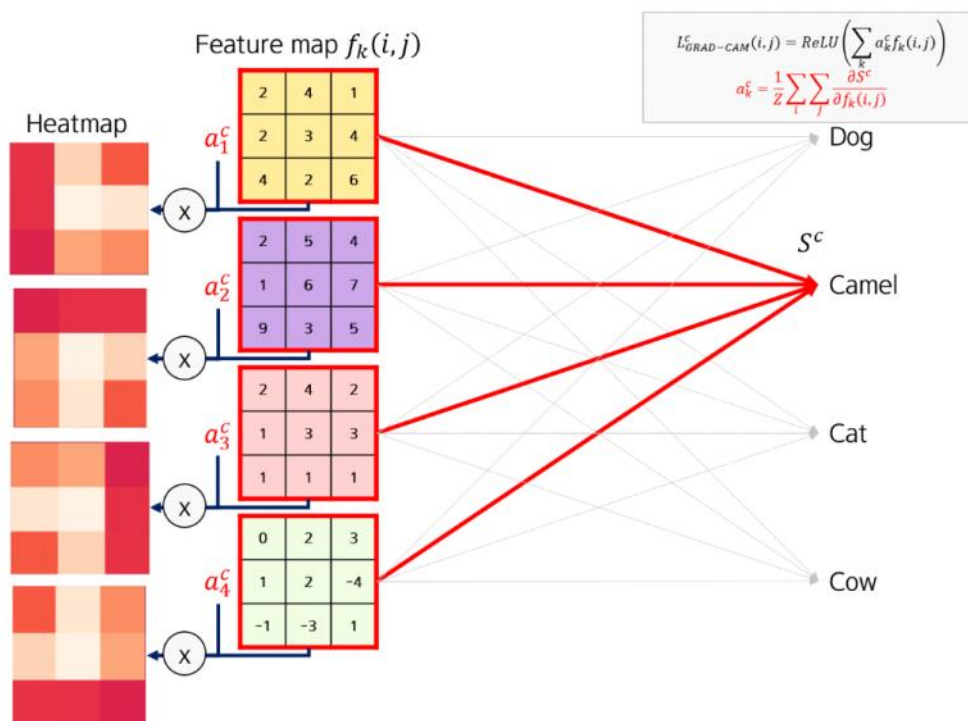


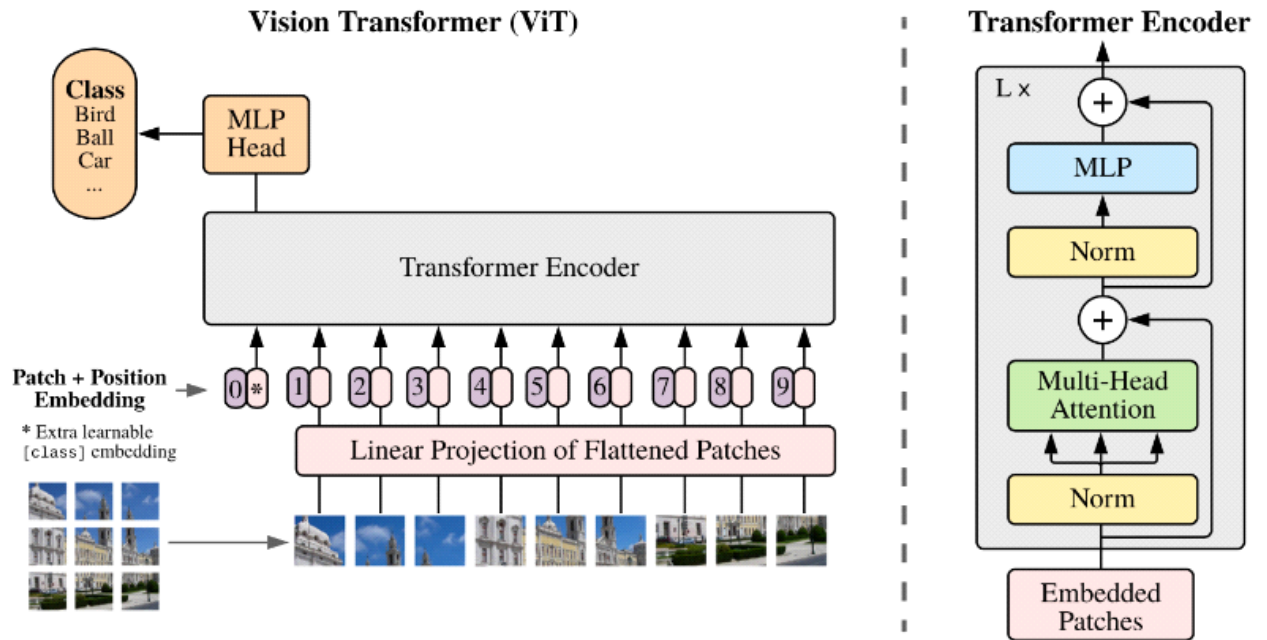




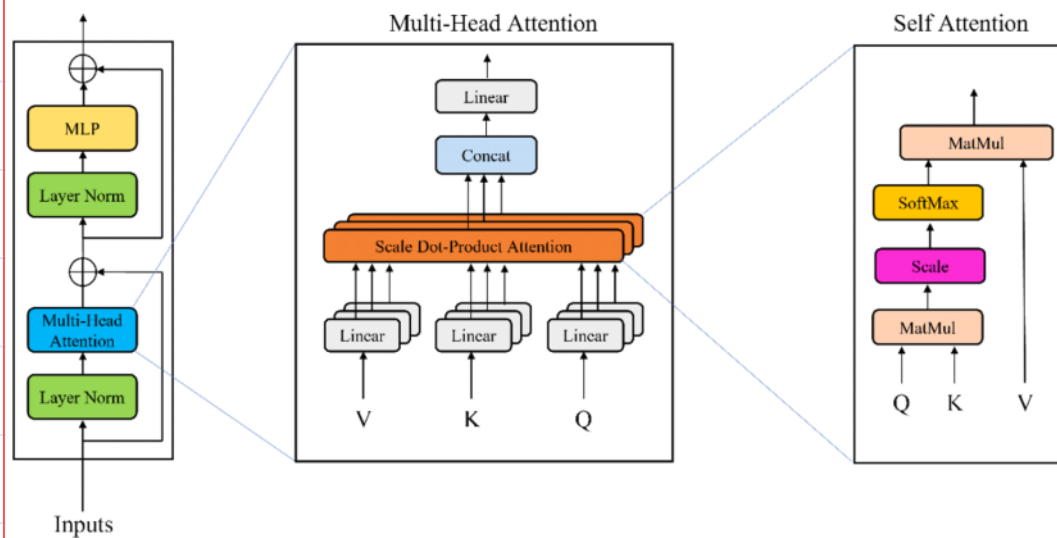
$$L_{Grad-CAM}^c(i, j) = ReLU\left(\sum_k a_k^c f_k(i, j)\right)$$

$$a_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial S^c}{\partial f_k(i, j)}$$





Transformer



Layer Norm: 입력 데이터를 정규화(normalization)하여 모델이 안정적으로 학습할 수 있도록 도움

Multi-Head Attention: 입력 데이터를 기반으로 중요한 특징(feature)을 추출

MLP (Multi-Layer Perceptron): Attention의 출력값을 비선형적으로 변환하여 더 복잡한 패턴을 학습

잔차 연결 (Residual Connection): 이전 입력값을 다음 레이어에 추가하여 학습 안정성을 높임

Multi-Head Attention

Multi-Head Attention은 Self-Attention을 여러 번 병렬로 계산하여 다양한 문맥 정보를 학습하는 메커니즘